

Модуль «Конструктор базы знаний» — оценка, план и архитектура

Дата: 03.07.2026. Основание: файл «Конструктор базы знаний Декомпозиция.xlsx».

1. Резюме

Вопрос	Ответ
Базовая команда (1 аналитик, 1 дизайнер, 1 BE, 1 FE)	~14–16 месяцев до полного скоупа
Ускоренный вариант (расширение до 2 BE + 2 FE + QA)	~8–9 месяцев
Оптимальный вариант (3 BE + 2 FE + QA + 0,5 DevOps)	~5,5–6,5 месяцев
Умеет ли GigaChat эмбединги	Да. Модели Embeddings, Embeddings-2, EmbeddingsGigaR через POST /api/v1/embeddings. EmbeddingsGigaR: вектор 2560 координат, контекст 4096 токенов, 14 ₽/1 млн токенов. Качество для русского языка — одно из лучших на рынке (Giga-Embeddings — топ-2 бенчмарка ruMTEB)

Ключевой вывод: узкое место базовой команды — один бэкенд-разработчик: на нём ~60–70 человеко-недель работ при ~45–50 на фронтенде. Наибольший эффект даёт добавление именно бэкендеров (один — со специализацией на RAG/LLM).

Допущения и открытые вопросы

- В Excel-файле строки 1–29 пусты, и список требований начинается с середины фразы («Асинхронную обработку операций...»). Похоже, верхняя часть декомпозиции потерялась. Оценка сделана по доступным строкам 30–85 + разумные допущения по «шапке» (нефункциональные требования, интеграция с платформой). Если найдётся полная версия — оценку нужно уточнить.
- Модуль работает на «технологической платформе» заказчика с «личным кабинетом пользователя» → предполагаем, что **аутентификация/SSO даёт платформа**, мы делаем авторизацию (роли, группы, права) внутри модуля.
- GigaChat у заказчика — предполагаем доступ через GigaChat API (облако Сбера или on-prem установка с тем же API). Если у заказчика собственная on-prem инсталляция — коннектор тот же, меняется только endpoint и способ аутентификации.

4. Нагрузка — до ~1–3 тыс. пользователей, десятки баз знаний (влияет на ресурсы, п. 6).
Требования ИБ (ГОСТ, аттестация, КИИ) в оценку **не включены** — при их наличии добавляется отдельный трек.

2. Скоуп (из декомпозиции)

Модуль позволяет создавать и запускать на платформе приложения двух типов:

- **База знаний типа 1** — справочная система из связанных ссылками блоков информации (wiki-подобная, со структурами данных, конструируемыми без кода).
- **База знаний типа 2** — диалоговая система «вопрос–ответ» на естественном языке с применением LLM и **RAG** (Retrieval Augmented Generation).

Функциональные блоки:

#	Блок	Роль
A	Администрирование: группы пользователей, автодоступ по группам, роли, выделение/высвобождение ресурсов под БЗ, глоссарий, категории, слияние тегов, журнал событий	Админ
B	Конструктор: структура данных (тип 1) / форма хранения (тип 2), модель доступа, привязка UI-элементов к данным, выбор и редактирование макетов интерфейса — преимущественно графическими инструментами (low-code)	Разработчик БЗ
C	Контент: гибридный редактор (код, таблицы, диаграммы), черновики с автосохранением, рецензирование (workflow отправки/одобрения), версионность (история, diff, откат), иерархия связей, теги, семантические связи	Разработчик БЗ
D	Медиа: загрузка изображений/видео/аудио, транскодирование под канал связи, медиатека с переиспользованием, пакетная загрузка, экспорт в PDF/TIFF и печатные сборники	Разработчик БЗ
E	Потребление: полнотекстовый поиск (морфология, опечатки), векторный поиск (тип 2), фильтры, рекомендации по истории просмотров, чат с LLM, настройки отображения, запросы на создание/обновление контента, рейтинг 1–5, геоинформационное представление, комментарии	Пользователь
F	Платформенное: асинхронная обработка (конвертация, индексация) через очереди, публичный REST API, уведомления о событиях, регистрация событий (кто, когда, операция, объект, результат)	Все

3. Архитектура

3.1. Общая схема

```
flowchart TB
    subgraph Клиенты
        UI[Web-приложение SPA<br/>React + TypeScript]
        EXT[Внешние системы]
    end

    subgraph Ядро["Бэкенд — модульный монолит + воркеры"]
        GW[API Gateway / BFF<br/>REST + SSE/WebSocket]
        CORE[Ядро: конструктор структур,<br/>контент, версии, RBAC, аудит]
    end
```

```

SEARCH[Сервис поиска<br/>гибридный: FTS + вектор]
RAG[RAG-оркестратор<br/>чанкинг, retrieval, генерация]
GIGA[Коннектор GigaChat<br/>chat + embeddings, OAuth, rate limit]
WRK[Воркеры: транскодирование ffmpeg,<br/>конвертация, индексация, экспорт PDF/TIFF]
NOTIF[Уведомления]

end

subgraph Хранилища
  PG[(PostgreSQL + pgvector<br/>+ PostGIS)]
  OS[(OpenSearch<br/>морфология RU, fuzzy)]
  S3[(S3 / MinIO<br/>медиа и файлы)]
  RDS[(Redis<br/>кэш, сессии)]
  MQ[[RabbitMQ / Kafka<br/>очереди задач]]
end

LLM[GigaChat API<br/>заказчика]

UI --> GW
EXT --> GW
GW --> CORE
GW --> SEARCH
GW --> RAG
CORE --> PG
CORE --> S3
CORE --> MQ
CORE --> RDS
MQ --> WRK
WRK --> OS
WRK --> S3
SEARCH --> OS
SEARCH --> PG
RAG --> SEARCH
RAG --> GIGA
GIGA --> LLM
CORE --> NOTIF

```

3.2. Технологии и подходы

Слой	Выбор	Почему
Фронтенд	React + TypeScript , Vite; UI-кит Ant Design/Mantine; конструктор макетов — drag-and-drop (dnd-kit) + JSON-схема макета; редактор — TipTap (ProseMirror) с расширениями код/таблицы/диаграммы (Mermaid); граф связей — Cytoscape.js; карты — Leaflet/MapLibre	TipTap — де-факто стандарт для гибридных редакторов, огромная экосистема расширений; JSON-схема макета — основа low-code
Бэкенд	Модульный монолит (Java/Kotlin + Spring Boot или Python + FastAPI — по компетенциям команды) + отдельные воркеры на очередях	Микросервисы для команды 1–3 BE — самоубийство по срокам; модульный монолит с чёткими границами модулей позволит выделить сервисы позже

Слой	Выбор	Почему
Данные конструктора	EAV/JSONB-модель в PostgreSQL : «структура данных» БЗ типа 1 описывается метасхемой (JSON Schema), записи — JSONB с валидацией	Даёт по-code создание структур без миграций БД; ресурсы «на базу знаний» = логическая изоляция схемой/тенантом
Полнотекстовый поиск	OpenSearch (или Elasticsearch): русская морфология (стеммер + hunspell), fuzzy для опечаток, фасетные фильтры	Требования «морфология + опечатки» из коробки
Векторный поиск	pgvector (HNSW-индекс) в той же PostgreSQL; альтернатива при росте — Qdrant	Меньше инфраструктуры; 2560-мерные векторы EmbeddingsGigaR pgvector поддерживает; гибридный поиск = RRF-слияние результатов OpenSearch + pgvector
Медиа	S3-совместимое хранилище (MinIO on-prem); транскодирование — воркеры с ffmpeg , несколько профилей качества под канал связи; выдача через presigned URL	
Экспорт	PDF — рендер HTML→PDF (Playwright/WeasyPrint), TIFF — конвертация из PDF (Ghostscript/ImageMagick); сборники — конкатенация по дереву разделов	
Очереди	RabbitMQ (достаточно; Kafka — только если платформа заказчика уже на ней)	Требование «асинхронная обработка через очереди» — явно в ТЗ
Доступ	RBAC: роли (админ/разработчик БЗ/пользователь) + группы + ACL на уровне БЗ и сущностей; модель доступа БЗ типа 1 конфигурируется разработчиком БЗ; интеграция с ЛК платформы по OIDC/SAML	
Аудит	Append-only журнал в PostgreSQL (партиционирование по датам) + выгрузка CSV/JSON; фиксируется: время, ID и ФИО пользователя, операция CRUD, объект, результат	Ровно по требованию из ТЗ
Геоданные	PostGIS + GeoJSON-слои на карте	
Рекомендации	v1: контентные (теги/категории просмотренного + популярность + рейтинг); ML-модели не требуются	Достаточно для требования, дешево
CI/CD, эксплуатация	Docker/Compose → Kubernetes (если есть у заказчика); GitLab CI; мониторинг Prometheus + Grafana, логи Loki/ELK	

3.3. Коннектор к GigaChat

GigaChat закрывает **обе** потребности — генерацию и эмбединги:

- **Эмбединги: да, умеет.** `POST /api/v1/embeddings`, модели `Embeddings`, `Embeddings-2` и `EmbeddingsGigaR` (рекомендуется для RAG): вектор **2560 координат**, контекст **4096 токенов**, цена 14 ₽/1 млн токенов (одинакова для всех моделей эмбедингов). `EmbeddingsGigaR` поддерживает **инструкции для retrieval-задач** (инструкция добавляется к поисковому запросу, но не к документам) — это заметно повышает точность поиска. По качеству на русском семейство Giga-Embeddings входит в топ бенчмарка `ruMTEB` (~2-е место, ~69

баллов) — для русскоязычной базы знаний это сильный выбор, отдельная embedding-модель не нужна.

- **Генерация:** `POST /api/v1/chat/completions`, поддерживается streaming (SSE) — нужен для «печатающегося» ответа в чате.

Что закладываем в коннектор:

1. **OAuth-аутентификация:** получение access token по Authorization key (scope `GIGACHAT_API_PERS/B2B/CORP`), токен живёт 30 мин → автообновление, кэш в Redis.
2. **Сертификаты НУЦ Минцифры** в trust store (облачный GigaChat работает по ним). Для on-prem инсталляции — конфигурируемый endpoint/mTLS.
3. **Rate limiting и очередь запросов** (лимиты параллельных запросов у GigaChat API), retry с backoff, circuit breaker.
4. **Абстракция провайдера** (интерфейс `LLMProvider / EmbeddingProvider`): GigaChat — первая реализация; если качество эмбеддингов на доменных текстах окажется недостаточным, подключаем локальную модель (bge-m3, multilingual-e5) без переписывания пайплайна. API GigaChat близок к OpenAI-формату, что упрощает абстракцию.
5. **Учёт токенов** по базам знаний — для контроля затрат заказчика.

⚠ **Риск для проверки на пилоте:** совокупная точность RAG-ответов зависит не только от эмбеддингов, но и от качества чанкинга и генеративной модели GigaChat на доменных документах. Закладываем в план **PoC RAG-пайплайна на реальных документах заказчика на раннем этапе** (см. фазу 2) с метриками качества (retrieval hit-rate, оценка ответов ассессорами).

3.4. RAG-пайплайн (БЗ типа 2)

Документы → парсинг (Apache Tika/Unstructured: docx, pdf, html...)
→ нормализация и чанкинг (по структуре документа, 500-1000 токенов, overlap)
→ EmbeddingsGigaR (батчами, асинхронно через очередь)
→ pgvector (HNSW) + параллельная индексация в OpenSearch

Запрос пользователя → эмбеддинг запроса (с retrieval-инструкцией)
→ гибридный поиск (вектор + FTS, слияние RRF, top-k)
→ фильтрация по правам доступа пользователя (!)
→ сборка промпта с контекстом и цитатами
→ GigaChat (streaming) → ответ со ссылками на источники
→ журналирование диалога, обратная связь (👍/👎)

4. Оценка трудоёмкости

Оценки в человеко-неделях (чел.-нед.), включают код-ревью и написание тестов разработчиком; **не** включают отпуска/больничные и внешние задержки (доступы, стенды заказчика) — на это заложен общий буфер 15 %.

#	Блок работ	BE	FE	Аналитик	Дизайнер
0	Каркас: CI/CD, окружения, скелет приложения, интеграция с ЛК платформы (SSO)	6	3	2	1
1	RBAC, группы, автодоступ, админка ролей и ресурсов БЗ	6	4	3	2
2	Конструктор структур данных (тип 1): метасхемы, JSONB-записи, модель доступа	9	6	4	3
3	Конструктор интерфейса: макеты, привязка элементов к данным, библиотека компонентов	4	9	3	5
4	Гибридный редактор (код/таблицы/диаграммы), черновики, автосохранение	3	8	2	3
5	Версионность (история, diff, откат) + рецензирование (workflow)	5	4	2	2
6	Медiateка: загрузка, транскодирование, пакетная загрузка, переиспользование	6	4	2	2
7	Полнотекстовый поиск (морфология, опечатки), фильтры, индексация через очереди	4	2	1	1
8	Коннектор GigaChat (chat + embeddings) + PoC качества RAG	4	—	1	—
9	RAG: ingestion-пайплайн, гибридный поиск, чат с LLM, цитирование, права	7	4	2	2
10	Иерархия связей, теги, семантические связи, глоссарий, слияние тегов	4	3	1	1
11	Рекомендации по истории просмотров	2	1	1	—
12	Экспорт PDF/TIFF, печатные сборники	4	1	1	1
13	Геоинформационное представление	2	3	1	1
14	Комментарии, рейтинги, запросы на создание/обновление контента, настройки отображения	3	4	1	2
15	Уведомления о событиях	2	1	1	—
16	Журнал событий (аудит) + просмотр и выгрузка	2	1	1	—
17	Публичный REST API + документация (OpenAPI)	3	—	1	—
18	Стабилизация, интеграционное тестирование, приёмка, документация	6	5	4	1
Итого		82	63	34	27
Итого с буфером 15 %		≈94	≈72	≈39	≈31

Аналитик и дизайнер не на критическом пути (34 и 27 чел.-нед. размазаны по всему сроку — они успевают работать «на опережение» спринта разработки).

5. Сроки и варианты состава команды

Вариант А — базовая команда (как есть): 1 аналитик, 1 дизайнер, 1 BE, 1 FE

Критический путь — бэкенд: ~94 чел.-нед. ≈ **21–22 месяца** чистой последовательной работы одного BE. Часть задач FE не блокируется BE, но фронтенд (72 нед. ≈ 16,5 мес.) тоже не помещается в год.

Срок: ~14–16 месяцев возможен только при урезании скоупа первой очереди (перенос экспорта TIFF, геоданных, рекомендаций, части конструктора UI во «вторую очередь»). Полный скоуп одной парой разработчиков — **~20–22 месяца**. Дополнительные риски: нулевой bus-фактор, некому делать код-ревью, тестирование ложится на аналитика.

Честная рекомендация: для этого скоупа вариант А не подходит.

Вариант Б — умеренное расширение: +1 BE, +1 FE, +1 QA (итого 7 человек)

Состав: 1 аналитик, 1 дизайнер, 2 BE, 2 FE, 1 QA.

- BE: 94 / 2 ≈ 47 нед.; FE: 72 / 2 ≈ 36 нед. Потери на координацию ~10 %.
- **Срок: ~8–9 месяцев** до полного скоупа. QA снимает тестирование с аналитика и заметно сокращает фазу стабилизации.

Вариант В — оптимальный: 9–10 человек (рекомендуется)

Состав: 1 аналитик, 1 дизайнер, 3 BE (один — с опытом RAG/LLM: пайплайн, качество поиска, коннектор), 2 FE, 1 QA, 0,5 DevOps; тимлид — играющий, из числа BE.

- BE: 94 / 3 ≈ 31–33 нед. с учётом координации; FE: ~36 нед. → фронтенд становится критическим путём, поэтому при желании ужаться до 5 мес. добавляется 3-й FE.
- **Срок: ~5,5–6,5 месяцев** до полного скоупа.

Дальнейшее расширение (4+ BE) почти не ускоряет: задачи связаны (конструктор → редактор → версии → поиск → RAG), закон Брукса начинает работать против.

Сводка

Вариант	Состав	Срок (полный скоуп)	Комментарий
А	1А + 1Д + 1BE + 1FE	~20–22 мес. (14–16 — только с урезанным скоупом)	Не рекомендуется: bus-фактор, нет QA
Б	1А + 1Д + 2BE + 2FE + 1QA	~8–9 мес.	Хороший баланс цены и срока
В	1А + 1Д + 3BE (1 с RAG) + 2FE + 1QA + 0,5 DevOps	~5,5–6,5 мес.	Рекомендуемый; +3-й FE → ~5 мес.

Фазирование (для варианта В, ~6 мес.)

Фаза	Срок	Содержимое	Результат
1. Фундамент	Мес. 1–2	Каркас, SSO, RBAC, группы, админка; конструктор структур данных; базовый редактор. Параллельно: PoC RAG на документах заказчика (коннектор GigaChat, эмбединги, метрики качества)	Демо БЗ типа 1 (минимум); вердикт по качеству GigaChat-эмбедингов
2. БЗ типа 1	Мес. 2–4	Конструктор UI, версии, рецензирование, медиатека с транскодированием, полнотекстовый поиск, теги и связи	БЗ типа 1 готова к пилоту
3. БЗ типа 2 (RAG)	Мес. 3–5	Ingestion-пайплайн, гибридный поиск, чат с цитированием и правами доступа, уведомления, REST API	БЗ типа 2 готова к пилоту
4. Полировка	Мес. 5–6	Рекомендации, гео, экспорт PDF/TIFF, комментарии/рейтинги, аудит-выгрузки, нагрузочное тестирование, приёмка	Релиз

6. Требуемые ресурсы

6.1. Инфраструктура (продуктив, до ~1–3 тыс. пользователей)

Компонент	Конфигурация	Кол-во
Бэкенд-приложение	4 vCPU / 8 GB	2–3 инстанса
Воркеры (ffmpeg, индексация, экспорт)	4 vCPU / 8 GB	2 (масштабируются по очереди)
PostgreSQL (+pgvector, PostGIS)	8 vCPU / 32 GB / SSD 500 GB+	primary + replica
OpenSearch	4 vCPU / 16–32 GB	2–3 узла
MinIO (S3)	4 vCPU / 8 GB	объём = медиаконтент × ~1,5 (профили транскодирования); старт от 1–2 TB
Redis, RabbitMQ	2 vCPU / 4 GB	по 2 (HA)
Мониторинг/логи	4 vCPU / 16 GB	1

Итого продуктив ориентировочно **~40–50 vCPU / 150–200 GB RAM** + сторидж по объёму контента; плюс тестовый стенд ~40 % от продуктива. **GPU не требуется** — вся ML-нагрузка на стороне GigaChat API. (Если заказчик захочет on-prem GigaChat — это его инфраструктура, обычно A100/H100-класс, в наш контур не входит.)

Оценка векторного хранилища: 1 чанк ≈ 700 токенов; 100 тыс. чанков (≈ 20–30 тыс. страниц документов) × 2560 float32 ≈ **1 GB** векторов + индекс — для pgvector незначительно.

6.2. Затраты на GigaChat API (ориентир)

- Индексация: 100 тыс. чанков × ~700 токенов ≈ 70 млн токенов × 14 ₽/млн ≈ **~1 000 ₽ разово** (переиндексация — по мере обновления контента). Эмбединги практически бесплатны.
- Генерация: основная статья — chat completions; зависит от тарифа заказчика и трафика чата (при 1 000 диалогов/день по ~3 000 токенов ≈ 90 млн токенов/мес; стоимость — по тарифному плану заказчика на GigaChat).

6.3. Люди (вариант В)

Роль	Загрузка	Период
Аналитик	100 %	весь проект
Дизайнер	100 % (к концу ~50 %)	весь проект
BE ×3 (1 — RAG/LLM)	100 %	весь проект
FE ×2	100 %	весь проект
QA	100 %	со 2-го месяца
DevOps	50 %	весь проект

7. Основные риски

Риск	Влияние	Митигация
Неполная декомпозиция (пустые строки 1–29 файла)	Скрытый скоуп → рост оценки	Запросить полную версию; фиксация скоупа с заказчиком до старта

Риск	Влияние	Митигация
Качество RAG на доменных документах (сканы, таблицы, специфичная лексика)	Неудовлетворённость ответами чата	RoC на реальных документах в фазе 1; абстракция провайдера эмбедингов; гибридный поиск
Требования «конструктора без программирования» трактуются шире (полноценный no-code)	Блоки 2–3 могут удвоиться	Ранние прототипы конструктора с аналитиком и заказчиком; ограничение библиотекой готовых компонентов
Интеграция с платформой заказчика (ЛК, доступы, стенды)	Простои команды	Запросить доступы/контракты API до старта; mock-слой платформы
Лимиты/квоты GigaChat API у заказчика	Деградация чата под нагрузкой	Очередь запросов, кэширование, выяснить квоты на пресейле
Требования ИБ (аттестация, ГОСТ) появятся позже	+1–3 мес.	Явно зафиксировать в договоре как отдельный этап

Источники

- [GigaChat API — векторное представление текста \(эмбединги\)](#)
- [EmbeddingsGigaR — характеристики модели](#)
- [GigaChat API — POST /embeddings](#)
- [Модели GigaChat](#)
- [Обзор эмбедингов для русского языка, ruMTEB \(vc.ru\)](#)